

Market Risk Project Report

ESILV - Market Risk (2025–2026)

Authors

Matthieu HANNA GERGUIS & Renaud DE L'EPINE

Group

A4-IF3

Module Coordinator

Matthieu GARCIN

Professor

Nicolas PESCI

December 30, 2025

Repository: [https:](https://github.com/MatthieuHannaGerguis/MarketRisk_Project_Hanna_Gerguis_DeLEpine)

[//github.com/MatthieuHannaGerguis/MarketRisk_Project_Hanna_Gerguis_DeLEpine](https://github.com/MatthieuHannaGerguis/MarketRisk_Project_Hanna_Gerguis_DeLEpine)

Contents

Question A - Historical VaR (Non-parametric)	2
I. Data preparation and return computation (before Question A.a)	2
II. Historical VaR estimation (Question A.a)	4
III. Out-of-sample validation (2017–2018) (Question A.b)	8
Question B - Expected Shortfall	10
Question C - Extreme Value Theory (EVT)	12
I. Data preparation and tail separation (EVT setup, before Question C.a)	12
II. Tail index estimation with the Pickands estimator (Question C.a)	12
III. EVT Value at Risk estimation (Question C.b)	16
Question D - Bouchaud Price Impact Model	18
I. Data preparation (before Question D)	18
II. Bouchaud price impact model: definition and parameters	20
III. Estimation strategy	21
Question E - Multiresolution Correlation & Hurst Exponent	26
I. Data Description and Preprocessing (Before Question E)	26
II. Multiresolution correlations using Haar wavelets (Question E.a)	28
III. Estimation of Hurst exponent and volatility scaling (Question E.b)	31
General conclusion (key results)	36

Project Overview

This report follows the same flow as our notebook: we go question by question and focus on the essentials: what we did, what we found, and how to read the results.

The notebook is meant to stand on its own. It is well commented, easy to follow, and (we hope) very complete. So if you want to double-check a step, rerun the code, or see extra outputs and plots, the notebook should explain everything on its own.

We aimed for the same standard in this report: a clean layout, clear results, and direct interpretations, while staying consistent with the notebook.

We also used Git and GitHub during the project to keep everything organized and track changes. The repository is here: https://github.com/MatthieuHannaGerguis/MarketRisk_Project_Hanna_Gerguis_DeLEpine

At the start of each question, we quickly state what data we use and, if needed, how we prepared it. This way, you immediately know what we are working with, without having to guess or scroll back.

We also state our conventions whenever it matters, so there is no confusion. For example, we always say how returns are defined, what we call a “loss”, which confidence level we use, and, when needed, the sign convention (like $\varepsilon \in \{-1, +1\}$ for trade direction).

Question A - Historical VaR (Non-parametric)

In Question A, we measure the market risk of the Natixis stock using a historical (non-parametric) Value-at-Risk (VaR) approach. We do not assume any parametric distribution for returns: we rely only on what the data tells us.

We use the file `Natixis_stock.txt`, which contains daily prices (date and price). We compute the VaR on a one-day horizon. The tail probability α is an input of the method and can be changed; in this report, we use $\alpha = 5\%$ (i.e., 95% confidence) for Question A.

We work with two time windows: (i) 2015-2016 is the in-sample period (to estimate the VaR), (ii) 2017-2018 is the out-of-sample period (to validate it in Question A.b).

Convention: We treat a negative return as a loss, so the VaR threshold is usually a negative return (left tail).

I. Data preparation and return computation (before Question A.a)

We first load the Natixis price data and compute daily returns. This is basic preprocessing, but it is very important. If dates are not read correctly, or if prices are not properly converted to numbers, the rest of the work can quickly go wrong.

We will reuse the exact same dataset and the exact same return series later (Questions B and C). So we clean the data and compute returns once, and we keep the same setup for the whole report.

Note: Prices are read from a `.txt` file, so the “Value” column is not automatically a float. In the raw file, decimals use a comma, so we replace commas with dots before converting to float.

```
[90]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#data = pd.read_csv("../data/Natixis_stock.txt", delimiter='\t',
#header=None, parse_dates=[0]) #we will mainly use main dataframe to analyze data in our whole project
data = pd.read_csv("../data/Natixis_stock.txt", delimiter='\t',
header=None, parse_dates=[0])
data.columns = ['Date', 'Value'] # rename columns
data['Date'] = pd.to_datetime(data['Date'], format="%d/%m/%Y")

data['Value'] = data['Value'].str.replace(',', '.').astype(float) #retype float number as we read a txt
data = data.sort_values('Date') # Let's sort by date to calculate returns

def Returns(data): #function to go to the next row in the same column(we didn't use pct_change() a pandas function)
    data['Returns'] = (data['Value'] - data['Value'].shift(1)) / data['Value'].shift(1)
    return data

data = Returns(data)
print(data)

#the first value of returns is NaN which is logical so we decided to erase the row from our dataset
data = data.dropna()
print(data)

alpha = 0.05 # tail probability / risk level (->95% confidence level)
value_portfolio = 100000 #value portfolio is the actual value of the portfolio
```

Figure 1: Natixis data preparation and computation of daily returns.

We now explain, what the code does.

Question A - Historical VaR (Non-parametric)

We load the text file and rename the two columns as **Date** and **Value**. Then we convert **Date** to a proper date format, since we will filter the sample by year later. We also make sure **Value** is a numeric float, because the raw file may use commas as decimal separators. Finally, we sort the dataframe by **Date** to keep the right time order. We then compute simple daily returns with the standard formula:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}.$$

```

      Date  Value  Returns
0  2015-01-02  5.621      NaN
1  2015-01-05  5.424 -0.035047
2  2015-01-06  5.329 -0.017515
3  2015-01-07  5.224 -0.019704
4  2015-01-08  5.453  0.043836
...      ...      ...      ...
1018 2018-12-21  4.045 -0.001481
1019 2018-12-24  4.010 -0.008653
1020 2018-12-27  3.938 -0.017955
1021 2018-12-28  4.088  0.038090
1022 2018-12-31  4.119  0.007583

[1023 rows x 3 columns]
      Date  Value  Returns
1  2015-01-05  5.424 -0.035047
2  2015-01-06  5.329 -0.017515
3  2015-01-07  5.224 -0.019704
4  2015-01-08  5.453  0.043836
5  2015-01-09  5.340 -0.020723
...      ...      ...      ...
1018 2018-12-21  4.045 -0.001481
1019 2018-12-24  4.010 -0.008653
1020 2018-12-27  3.938 -0.017955
1021 2018-12-28  4.088  0.038090
1022 2018-12-31  4.119  0.007583

[1022 rows x 3 columns]
```

Figure 2: Data output (with date, value and computed returns).

For this point, the dataset is ready. We have clean prices, clean dates, and one return per trading day. We also set $\alpha = 0.05$ and a portfolio value (here 100,000), since later we may express risk in monetary terms.

Bonus: As a quick extra check, we plot the Natixis stock price over time, just to see the overall pattern and make sure the series looks consistent.

```
[108]: plt.figure(figsize=(10, 5)) # simple visualization of the Natixis stock price over time
plt.plot(data["Date"], data["Value"])
plt.xlabel("Date")
plt.ylabel("Stock price")
plt.title("Natixis stock price evolution")
plt.show()
```

Figure 3: Code used to plot Natixis stock price evolution.

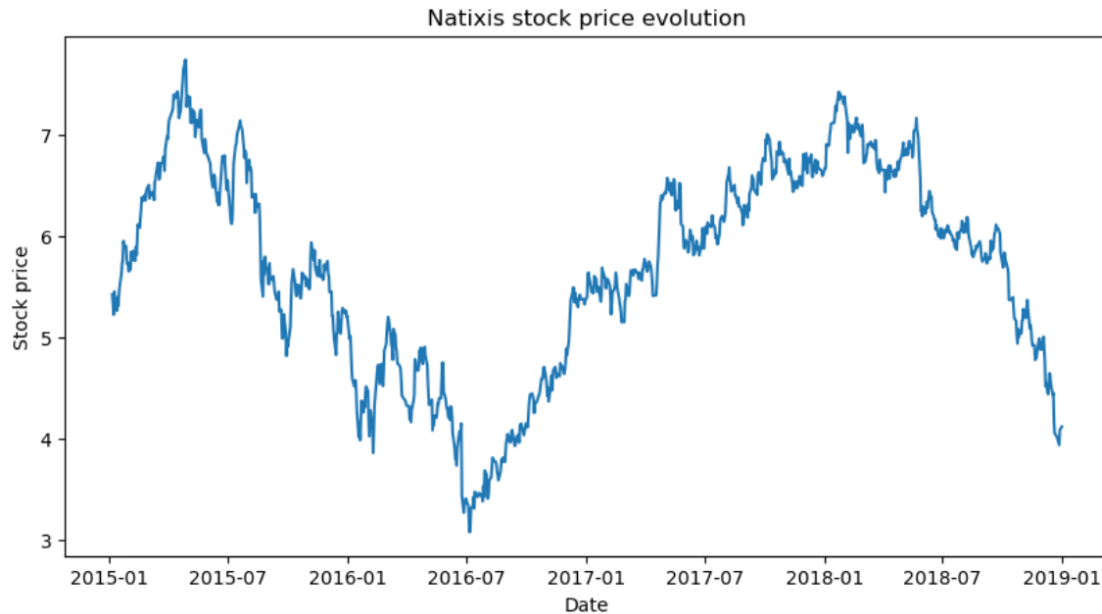


Figure 4: Natixis stock price evolution

Note: To clarify, this plot is not used to compute the VaR. It is only a sanity check: it helps us see that dates are in the right order and that the series looks like a normal price series.

II. Historical VaR estimation (Question A.a)

In this part, we estimate a one-day historical VaR for Natixis returns. We keep the same setup as in the Part I : we use daily simple returns, we take $\alpha = 0.05$ (95% confidence), and we use a portfolio value of 100,000 so we can also express the VaR in euros.

We follow the same logic as in the notebook: first, we isolate the in-sample period (January 2015 to December 2016), which we use to *estimate* the VaR. Then, in Question A.b, we use January 2017 to December 2018 to check it out-of-sample.

```
[9]: # on our sorted dataset we select the time periods required to estimate and validate the historical VaR.
data_date_sort_2015 = data[(data["Date"] >= '2015-01-01') & (data["Date"] <= '2016-12-31')]
data_date_sort_2017 = data[(data["Date"] >= '2017-01-01') & (data["Date"] <= '2018-12-31')]

#Once the selection is done now we can sort returns !
data_date_sort_2015 = data_date_sort_2015.sort_values("Returns") # using build in sort function of pandas
data_date_sort_2017 = data_date_sort_2017.sort_values("Returns")

print(data_date_sort_2015) # quick check up
```

Figure 5: Selecting the periods and sorting returns.

	Date	Value	Returns
378	2016-06-24	3.439	-0.171325
345	2016-05-10	4.083	-0.069083
358	2016-05-27	4.460	-0.061448
405	2016-08-02	3.422	-0.060664
277	2016-02-02	4.208	-0.058613
..	...	...	...
283	2016-02-10	4.101	0.062435
305	2016-03-11	5.083	0.063389
164	2015-08-25	5.755	0.064755
403	2016-07-29	3.685	0.075912
326	2016-04-13	4.785	0.078431

[512 rows x 3 columns]

Figure 6: 2015-2016 returns ranked from worst to best.

Course reference: We recall the definition of the historical Value at Risk as introduced during the lectures (see slide on Value at Risk, page 68).

Definition

Let X be a random variable depicting a gain. Then, the value at risk with confidence α is:

$$\text{VaR}_\alpha(X) = -F_X^{-1}(1 - \alpha).$$

Alternatively, if Y is a loss ($Y = -X$), $\text{VaR}_\alpha(Y) = F_Y^{-1}(\alpha)$.

Figure 7: Definition of the historical Value at Risk (Course slide page 68).

We now compute the historical VaR at level α . In the notebook, we start with the standard approach based on the empirical quantile. So we simply take the α -quantile of the observed returns.

If r is the daily return, the VaR threshold (in return terms) is:

$$\text{VaR}_\alpha^{(\text{ret})} = q_\alpha(r),$$

where $q_\alpha(r)$ is the empirical α -quantile.

Convention: returns can be negative, so $\text{VaR}_\alpha^{(\text{ret})}$ is usually negative (left tail threshold). If we want a positive loss number, we report $-\text{VaR}$ (and in euros, we multiply by the portfolio value).

```
[91]: # Historical VaR estimation using the empirical quantile
var_historical = np.quantile(data_date_sort_2015["Returns"], alpha)

var_historical_value = - var_historical * value_portfolio # we have VaR expressed in monetary value
var_historical, var_historical_value

[91]: (np.float64(-0.03777913198405718), np.float64(3777.9131984057176))
```

Figure 8: Computing the empirical VaR (returns and euros).

With $\alpha = 0.05$, the historical VaR on returns is about -0.03778. So the threshold is roughly -3.78% over one day. With a portfolio value of 100,000, this corresponds to a monetary VaR of about 3,777.91 euros.

What does this mean? Over 2015-2016, only 5% of the days have a return below -3.78%. So, with this historical definition, a "bad" 5% day means losing at least about 3,778 euros.

We now do the same thing, but with a smoother distribution. We fit a KDE to the returns, and then we take the same 5% quantile.

Course reference: This is the method shown in the course slides (VaR estimation, slide 69), and it is also what we use in the notebook.

We use the biweight kernel:

$$K(u) = \begin{cases} \frac{15}{16}(1 - u^2)^2 & \text{if } |u| \leq 1, \\ 0 & \text{otherwise.} \end{cases}$$

With this kernel, the KDE at a point x is:

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - r_i}{h}\right),$$

where r_i are the observed returns (2015-2016 here) and h is the bandwidth.

In the code, we set the bandwidth with a standard rule of thumb:

$$h = \left(\frac{4\hat{\sigma}}{3n}\right)^{1/5},$$

where $\hat{\sigma}$ is the sample standard deviation of returns and n is the number of observations.

```
[11]: def kernel(u): # we define the biweight kernel used in the non-parametric estimation
      if abs(u) <= 1:
          return (15/16) * (1 - u**2)**2
      else:
          return 0

      def kernel_density(data, x): # Kernel density estimation function where x is the point where the density is evaluated
          # bandwidth chosen
          h = ((4 * data['Returns'].std()) / (3 * len(data['Returns'])))**0.2
          n = len(data['Returns'])
          kernel_sum = 0

          # we sum the contribution of each observation to the kernel density
          for r in data['Returns']:
              kernel_sum += kernel((x - r) / h)
          return kernel_sum / (n * h) # final kernel density value

      def VaR_Kernel(data, alpha): # function to compute the non-parametric VaR using the kernel distribution
          probabilities = []
          for r in data['Returns']:
              probabilities.append(kernel_density(data, r))

          total = sum(probabilities) # normalization to ensure probabilities sum to one
          if total != 0:
              probabilities = [p / total for p in probabilities]
              sorted_returns = sorted(data['Returns']) # computation of the empirical quantile based on cumulative probabilities
              cumulative_prob = 0

              for i, p in enumerate(probabilities):
                  cumulative_prob += p
                  if cumulative_prob >= alpha:
                      return sorted_returns[i]

          Value_at_Risk = VaR_Kernel(data_date_sort_2015, 0.05)
          print(f"Value at Risk of Kernel : {Value_at_Risk}")

      Value at Risk of Kernel : -0.03478608556577359
```

Figure 9: KDE-based VaR: biweight kernel and 5% quantile.

The kernel-based VaR (return threshold) is:

$$\text{VaR}_{0.05, \text{kernel}}^{(\text{ret})} \approx -0.03479,$$

so roughly -3.48% over one day.

In euros (same portfolio value 100,000), this corresponds to a loss of about:

$$-\text{VaR}_{0.05, \text{kernel}}^{(\text{ret})} \times 100,000 \approx 3,478.61 \text{ euros.}$$

This is positive by convention.

Finally, the notebook plots the KDE together with the return histogram and the VaR cutoff (vertical line). This is a quick visual check to see where the 5% left tail begins.

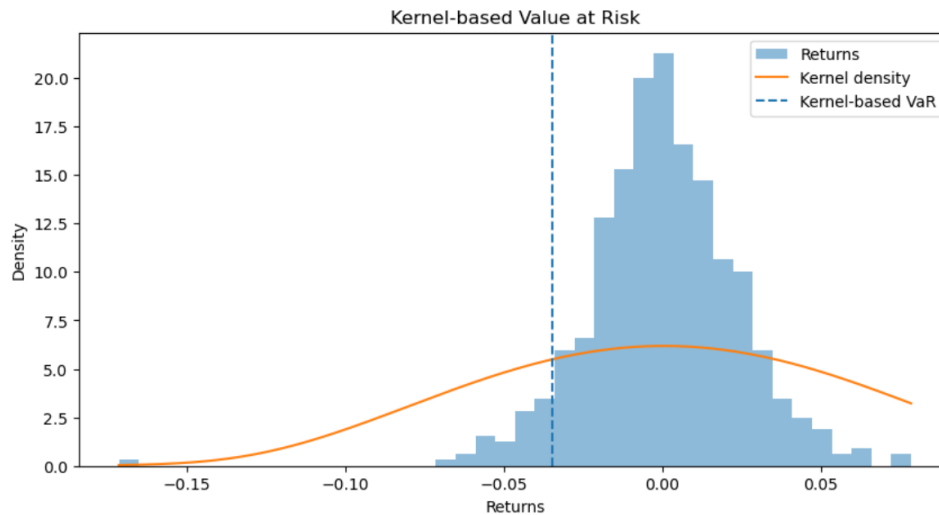


Figure 10: Return KDE and the 5% VaR threshold (2015-2016).

Remark: Here, the kernel VaR is slightly less extreme than the empirical VaR. This can happen because the KDE smooths the distribution, so the left tail is less "step-like". In Question A.b, we will check out-of-sample (2017-2018) whether this VaR level is still realistic.

III. Out-of-sample validation (2017–2018) (Question A.b)

We now check if the non-parametric VaR estimated on 2015-2016 behaves well on new data. We take the out-of-sample period 2017-2018 and count how often returns fall below the VaR threshold.

Convention: Here, an "exceedance" means a return below the VaR threshold, since the VaR on returns is a left-tail cutoff. So we count days where $r_t < \text{VaR}_\alpha$.

In the notebook, we do this in a simple loop: we take the 2017-2018 returns, count how many are below `Value_at_Risk` (the KDE VaR from Part II), and compute the proportion.

```
[13]: # We now look at how many returns exceed the VaR threshold on the 2017-2018 period

data_2017_2018 = data[
    (data['Date'] >= '2017-01-01') &
    (data['Date'] <= '2018-12-31') # out-of-sample period (2017-2018)
]
returns_2017_2018 = data_2017_2018['Returns']

exceedances = 0

for r in returns_2017_2018:
    if r < Value_at_Risk:
        exceedances += 1 # count how many times returns fall below the estimated VaR
exceedance_ratio = exceedances / len(returns_2017_2018) # proportion of exceedances

print("With non-parametric VaR")
print(exceedances, "returns out of", len(returns_2017_2018), "exceed the VaR threshold ")
print("This corresponds to", round(exceedance_ratio * 100, 2), "% for a risk level of 5%")

With non-parametric VaR
9 returns out of 510 exceed the VaR threshold
This corresponds to 1.76 % for a risk level of 5%
```

Figure 11: Counting exceedances in 2017-2018 (KDE VaR).

In our case, we get: 9 exceedances out of 510 returns, so an exceedance rate of 1.76%.

We compare this to the nominal level $\alpha = 5\%$. Since 1.76% is below 5%, we see fewer exceedances than what a 5% VaR would suggest. So, over 2017-2018, the kernel-based VaR looks conservative.

Note: This kind of historical or kernel VaR assumes that past return behavior (2015-2016 here) is still a good proxy for what comes next (2017-2018). In practice, this can fail, especially when volatility changes a lot, which is why this out-of-sample check matters.

Question B - Expected Shortfall

In this question, we compute the Expected Shortfall associated with the non-parametric VaR estimated in Question A.

We reuse the same return series as in Question A, and the same non-parametric VaR threshold (`Value_at_Risk`). So we do not re-estimate the VaR here. We only compute ES using the VaR level we already obtained.

The idea is simple. VaR gives a cutoff (a quantile of the return distribution). Expected Shortfall goes one step further: it looks at what happens inside the VaR tail, that is, when returns fall below the VaR threshold. So we take all returns below the VaR threshold and compute their average.

Sign convention: Here we work with returns (not losses). So the left-tail VaR and the ES are negative numbers. In practice, we select the tail with $R \leq VaR_\alpha$ (this is what the code does with `Returns <= Value_at_Risk`), and then ES is the average of these tail returns, so it is also negative. If we want a positive loss number, we report $-VaR$ and $-ES$ (and in euros, we multiply by the portfolio value).

Course reference: Expected Shortfall (also called AVaR or conditional VaR) is introduced in the lectures as the average outcome beyond VaR, in the tail. Here we work with returns, so we use the left-tail definition based on returns (see slide "Expected shortfall", page 75). In the lectures, Expected Shortfall is also presented as a spectral (distortion) risk measure. For a continuous distribution, it can be written as:

$$ES_\alpha = \mathbb{E}[R \mid R \leq VaR_\alpha].$$

This is the definition we use here (same as in the notebook).

Expected shortfall

Let X be the gain of the portfolio, $\alpha \in [0, 1]$ a confidence level. We define several risk measures depicting the average of the $1 - \alpha$ worst losses:

- the **worst conditional expectation**: $WCE_\alpha(X) = \sup\{\mathbb{E}[-X|A], \mathbb{P}(A) \geq 1 - \alpha\}$,
- the **tail conditional expectation**: $TCE_\alpha = \mathbb{E}[-X|X \leq -VaR_\alpha(X)]$,
- the **average VaR** (AVaR), or **expected shortfall** (ES), or **conditional VaR**:
 $ES_\alpha(X) = \frac{1}{1-\alpha} \int_\alpha^1 VaR_r(X) dr = \frac{1}{1-\alpha} \int_\alpha^1 F_{-X}^{-1}(r) dr.$

Figure 12: Expected Shortfall definition (Course slide page 75).

Computation of the Expected Shortfall:

In practice, we take all in-sample returns (2015-2016) that are below (or equal to) the non-parametric VaR, and we compute their mean. This is exactly what the notebook does.

```
[113]: # select the returns that are below (or equal to) the VaR threshold (worst tail)
      rdt_over_VaR = data_date_sort_2015["Returns"][data_date_sort_2015["Returns"] <= Value_at_Risk]
      # We compute ES empirically: we average the returns in the left tail (R <= VaR_alpha),
      # which matches ES_alpha = E[R|R <= VaR_alpha].
      ES = np.mean(rdt_over_VaR)

      print("Non parametric VaR:", Value_at_Risk)
      print("Expected Shortfall (non-parametric):", round(ES, 5))

Non parametric VaR: -0.03478608556577359
Expected Shortfall (non-parametric): -0.05004
```

Figure 13: code tail selection and ES computation.

From the notebook output, the non-parametric VaR (95%) is about -3.48% and the ES (95%) is about -5.00%. This is expected: ES averages the returns inside the worst 5% tail, so it is more extreme than VaR in magnitude ($|ES| > |VaR|$).

Question C - Extreme Value Theory (EVT)

In this question, we focus on the analysis of extreme returns using tools from Extreme Value Theory (EVT). We use the Natixis return series and study extreme gains and extreme losses separately, to get a clearer view of tail behavior.

We use the same return series as in Question A, but now we zoom in on the extreme tails with an EVT approach.

Sign convention: in this section we work in loss terms. When returns are negatif, we define losses = $-$ returns, so losses are positive numbers. This means the EVT VaR is a positive loss. If we want to compare with the previous sections (VaR on returns), we just take the negative of it.

I. Data preparation and tail separation (EVT setup, before Question C.a)

To apply Extreme Value Theory, we first split returns into two tails: one for extreme gains and one for extreme losses. We analyze them separately. For losses, we set $L_t = -r_t$, so losses are positive and EVT is applied to the right tail of the loss distribution.

In the notebook, we do it in the simplest possible way: we take positive returns as **gains**, and we convert negative returns into positive numbers for **losses**. Next we sort the two series, because EVT estimators use order statistics (sorted samples).

```
[15]: gains = data['Returns'][data['Returns'] > 0]
      losses = -data['Returns'][data['Returns'] < 0]# converts losses into positive values, thus EVT applies to the right tail.

      gains_sorted = np.sort(gains)
      losses_sorted = np.sort(losses)
```

Figure 14: Tail separation for EVT: gains ($R > 0$) and losses ($L = -R > 0$), then sorts the two

Note: With the transformation $L = -R$ (for $R < 0$), we apply EVT to the right tail of L (large losses). So the EVT-based VaR naturally comes as a positive loss threshold.

II. Tail index estimation with the Pickands estimator (Question C.a)

In this part, we estimate the extreme value index (shape parameter) ξ for the gain tail and for the loss tail. We use the Pickands estimator, which estimates ξ directly. We do not estimate the full GEV location and scale parameters, since we do not need them for Question C.b.

Course reference: We use the Pickands estimator from the EVT lectures to estimate the tail index ξ . The notebook follows the formula from the EVT-based VaR estimation slides (page 180).

then, the Pickands estimator, defined by:

$$\xi_{k(n),n}^P = \frac{1}{\log(2)} \log \left(\frac{X_{n-k(n)+1:n} - X_{n-2k(n)+1:n}}{X_{n-2k(n)+1:n} - X_{n-4k(n)+1:n}} \right)$$

Figure 15: Pickands estimator for the tail index ξ (course slide page 180).

Course reference: The sign of ξ tells us what type of tail we have. As recalled in the EVT lectures (page 166): $\xi > 0$ corresponds to a Fréchet distribution (heavy tails), $\xi = 0$ to a Gumbel distribution, and $\xi < 0$ to a Weibull distribution.

Remark

Then:

- *if $\xi > 0$, the GEV is of Fréchet kind,*
- *if $\xi = 0$, the GEV is of Gumbel kind,*
- *if $\xi < 0$, the GEV is of Weibull kind.*

Figure 16: Sign of ξ (Fréchet / Gumbel / Weibull) (course slide page 166).

We now apply the Pickands estimator separately to extreme gains and to extreme losses. We use the sorted samples built in Part I: `gains_sorted` and `losses_sorted`.

In the notebook, we write a small function `pickands_estimator(extreme_values)`. We set $k = \lfloor \log(n) \rfloor$ (rule of thumb from the lectures) to choose how many tail points we use. Then we take the needed order statistics and compute $\hat{\xi}$ with the Pickands formule.

```
[98]: def pickands_estimator(extreme_values):

    #Pickands estimator for the tail index, extreme observations sorted in increasing order.
    n = len(extreme_values)

    # we set k = int(log(n)) (rule of thumb from the lectures) to choose how many tail points we use.
    k = int(np.log(n))
    x1 = extreme_values[n - k] #required order statistics
    x2 = extreme_values[n - 2 * k]
    x3 = extreme_values[n - 4 * k]

    # we seek the Pickands estimator according to the theoretical formula
    xi_hat = (1 / np.log(2)) * np.log((x1 - x2) / (x2 - x3))
    return xi_hat

# we estimate the tail index separately for gains and for losses
xi_gains = pickands_estimator(gains_sorted)
xi_losses = pickands_estimator(losses_sorted)
xi_gains, xi_losses

[98]: (np.float64(0.5772338569463286), np.float64(-0.508971577934174))
```

Figure 17: Pickands estimator: ξ for gains and losses (code and output).

From the notebook, we get a positive tail index for extreme gains ($\xi \approx 0.58$) and a negative tail index for extreme losses ($\xi \approx -0.51$). So the two tails do not behave the same way, which points to an asymmetric pattern in Natixis returns.

Using the lecture interpretation: $\xi > 0$ for gains suggests heavy-tailed behavior (Fréchet-type extremes), while $\xi < 0$ for losses suggests lighter tails (Weibull-type extremes).

Notes: In standard EVT applications, we often assume returns are independent and identically distributed. Also, the Pickands estimator uses only a limited number of extreme observations, so the estimate can be sensitive to sampling variability and to the choice of the intermediate sequence k .

Bonus: we plot the empirical distributions of extreme gains and extreme losses. This helps us see the tail difference suggested by the Pickands estimates.

```
[114]: # plot
plt.figure(figsize=(8, 5))

plt.hist(gains_sorted, bins=40, density=True, alpha=0.7,
        label="Extreme gains", edgecolor="black")

plt.hist(losses_sorted, bins=40, density=True, alpha=0.7,
        label="Extreme losses", edgecolor="black")

plt.xlabel("Return magnitude")
plt.ylabel("Density")
plt.title("Empirical distribution of extreme gains and losses")

plt.legend()
plt.grid(True)
plt.show()
```

Figure 18: Code: histogram comparison of extreme gains and extreme losses.

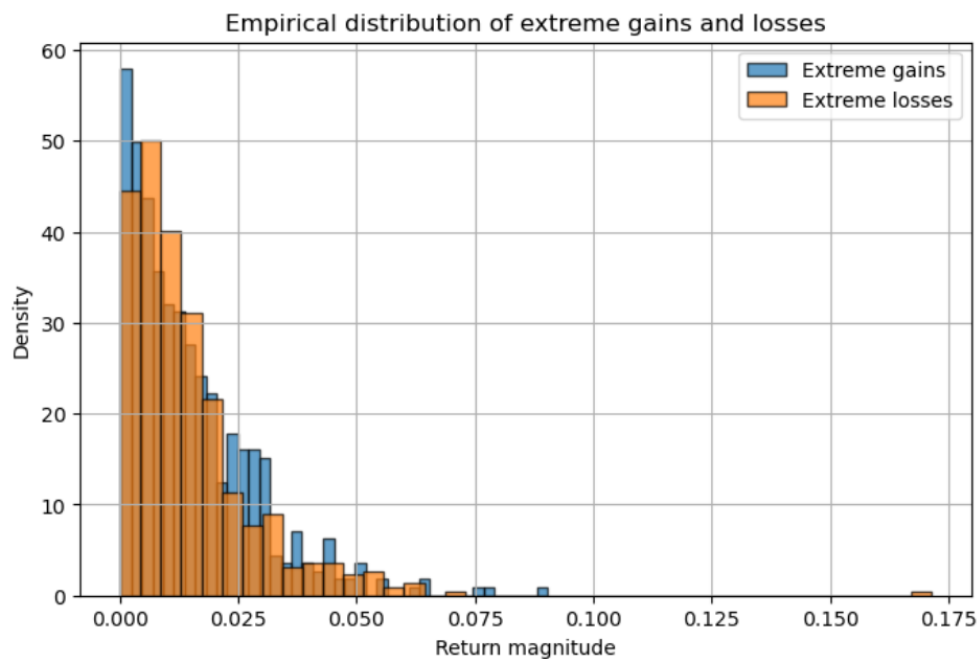


Figure 19: Extreme gains vs extreme losses (empirical distributions).

Comment: the empirical distributions confirm the asymmetrical tail behavior (extreme gains more pronounced than extreme losses).

III. EVT Value at Risk estimation (Question C.b)

In this part, we use Extreme Value Theory to compute Value at Risk estimates from the tail behavior we studied before with the Pickands estimator. We focus on the left tail of returns (losses), since VaR is a downside risk measure. We compute the EVT-based VaR for several confidence levels.

We compute EVT VaR using the tail index estimate from Question C.a. As in standard EVT settings, we assume returns are independent and identically distributed (iid).

Course reference: We use the EVT VaR formula from the lectures, where VaR is written using the Pickands estimate of the tail index ξ (see EVT-based VaR estimation slides, page 187).

Pickands estimator

$$VaR(p) = \frac{\left(\frac{k}{n(1-p)}\right)^{\xi^P} - 1}{1 - 2^{-\xi^P}} (X_{n-k+1:n} - X_{n-2k+1:n}) + X_{n-k+1:n},$$

where ξ^P is the Pickands estimator of the GEV parameter.

Figure 20: EVT VaR formula using the Pickands estimator (Course slide page 187).

We now compute the EVT Value at Risk for several confidence levels and compare it to the historical VaR obtained in Question A.

Convention: As above we work with losses $L = -R$ (for $R < 0$), so EVT VaR is reported as a positive loss threshold.

In the notebook, we implement a function `var_evt_pickands(extreme_losses, xi_hat, alpha)`. We keep the same choice of $k = \lfloor \log(n) \rfloor$ to stay consistent with Question C.a. Then we use the needed order statistics and the EVT formula from the course to compute VaR at a given confidence level

Notation: In this part, the variable `alpha` in the code is the *confidence level* (for example 0.99), so the tail probability is $1 - \alpha$. This is why the EVT formula includes the term $(1 - \alpha)$.

```
[101]: def var_evt_pickands(extreme_losses, xi_hat, alpha):

    n = len(extreme_losses)
    # we keep the same choice of k to ensure consistency between the two steps
    k = int(np.log(n))
    # tail order statistics required by the EVT formula
    x_k = extreme_losses[n - k]
    x_2k = extreme_losses[n - 2 * k]

    numerator = (k / (n * (1 - alpha))) ** xi_hat - 1
    denominator = 1 - 2 ** (-xi_hat)
    var_evt = (numerator / denominator) * (x_k - x_2k) + x_k
    return var_evt
```

Figure 21: Code: EVT VaR with Pickands and order statistics.

We then compute EVT VaR for several confidence levels (as in the notebook: 90%, 95%, and 99%). We apply the formula to `losses_sorted` and use the tail index estimate for losses (`xi_losses`).

```
[102]: confidence_levels = [0.90, 0.95, 0.99] # the several confidence levels

for alpha in confidence_levels:
    var_alpha = var_evt_pickands(losses_sorted, xi_losses, alpha)
    print(f"EVT-based VaR (Pickands) at {int(alpha*100)}% confidence level: {var_alpha:.5f}")

EVT-based VaR (Pickands) at 90% confidence level: 0.03141
EVT-based VaR (Pickands) at 95% confidence level: 0.04354
EVT-based VaR (Pickands) at 99% confidence level: 0.05958
```

Figure 22: Pickands EVT VaR: results with 90%, 95%, and 99%.

We can see the VaR going up as we move to higher confidence levels: around 3.1% at 90%, 4.4% at 95%, and about 6.0% at 99%. This is what we expect: higher confidence means we go further into the tail, so losses are rarer but more larger.

Note: EVT-based VaR relies on extrapolation beyond the observed sample using only the tail behavior of the distribution. So the estimates become more sensitive when the confidence level increases.

Compared to the historical VaR from Question A, EVT VaR gives more importance to the tail of the loss distribution. So at very high confidence levels, it can give higher risk estimates. Also, EVT VaR is based on a limited number of extreme observations, so it can be sensitive to the choice of k and to sampling variability.

Question D - Bouchaud Price Impact Model

We now estimate the parameters of Bouchaud's price impact model using the TD4 framework.

For this question, we use the dataset provided for TD4 (`Dataset_TD4.xlsx`), not the Natixis price series used in Questions A-C. The goal here is to focus on market impact and liquidity. So the inputs are different, we work with intraday variables such as price, volume, bid-ask spread, and trade direction (the sign).

Note: in this first part, we keep preprocessing light. We mainly rename columns and do a few quick plots to check the data. We only do heavier cleaning later, when we actually need it for the estimation step (this is the notebook approach).

I. Data preparation (before Question D)

The objective is to obtain a clean, time-indexed table containing the key variables required by the model: a price variable, traded volume, bid-ask spread S , and the trade sign $\varepsilon \in \{-1, +1\}$.

We start by loading the dataset and inspecting its structure. This is just to see what the raw columns look like and to make sure the file is read correctly.

```
[20]: df_td4 = pd.read_excel("../data/Dataset_TD4.xlsx")
      df_td4.head()
```

	transaction date (1=1day=24 hours)	bid-ask spread	volume of the transaction (if known)	Sign of the transaction	Price (before transaction)
0	0.000202	0.1100	8.0	-1	100.000
1	0.001070	0.1030	NaN	1	99.984
2	0.001496	0.1015	NaN	-1	100.029
3	0.003336	0.0920	NaN	1	99.979
4	0.003952	0.1106	NaN	1	100.060

Figure 23: First rows of `Dataset_TD4.xlsx`.

Then we create a working dataframe and rename the columns. We do this to keep short, clear names that match the model notation used later. So it is easier to read the code and the formulas.

```
[21]: df_vars = df_td4.copy()

df_vars = df_vars.rename(columns={
    "transaction date (1=1day=24 hours)": "time",
    "Price (before transaction)": "price",
    "volume of the transaction (if known)": "volume",
    "bid-ask spread": "spread",
    "Sign of the transaction": "eps" # copy and rename columns without cleaning the dataset
})

df_vars.head()
```

```
[21]:
```

	time	spread	volume	eps	price
0	0.000202	0.1100	8.0	-1	100.000
1	0.001070	0.1030	NaN	1	99.984
2	0.001496	0.1015	NaN	-1	100.029
3	0.003336	0.0920	NaN	1	99.979
4	0.003952	0.1106	NaN	1	100.060

Figure 24: `df_vars` after renaming columns.

Convention: `eps` is the trade sign, with values in $\{-1, +1\}$ (sell vs buy direction). This is the direction variable used in the signed impact model.

Bonus : before estimating the model, we make two quick plots. These are simple sanity checks to see if the data looks normal.

First, we plot the price evolution over time. This helps us see if the price series behaves normally (no obvious data problem).

```
[22]: plt.figure(figsize=(8, 4))
plt.plot(df_vars["time"], df_vars["price"])
plt.xlabel("Time")
plt.ylabel("Price")
plt.title("Price evolution over time")
plt.show()
```

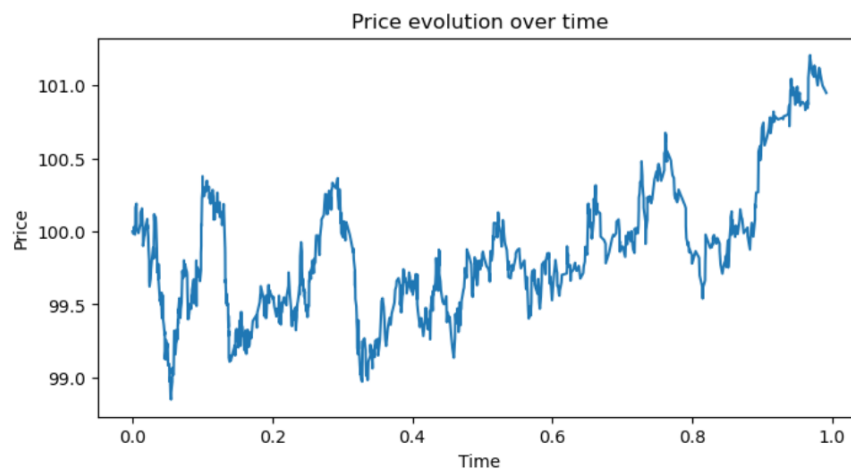


Figure 25: Price evolution over time.

Second, we look at the distribution of trade signs. This checks if buy and sell directions look roughly balanced, which is important before fitting a signed price impact model.

```
[23]: df_vars["eps"].value_counts().plot(kind="bar") # distribution of the trade sign to check balance between buys and sells
plt.title("Distribution of trade signs")
plt.xlabel("Trade sign")
plt.ylabel("Count")
plt.show()
```

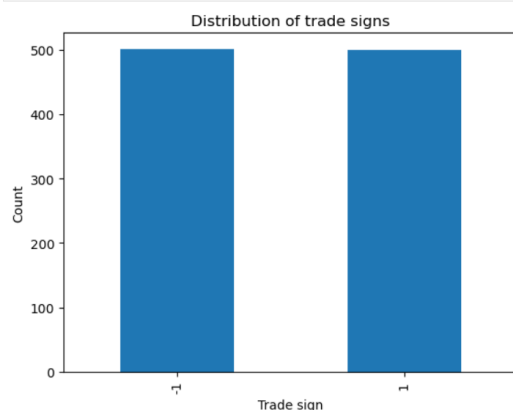


Figure 26: Distribution of trade signs (buy/sell).

From these plots, we see that trade signs look balanced. This is in line with what we expect before estimating a signed price impact model.

II. Bouchaud price impact model: definition and parameters

We now introduce Bouchaud's price impact model. We will use it to estimate how trades move prices.

The idea is simple: today's price is the result of many past trades. Each trade pushes the price up or down (because of the trade sign), and this impact slowly fades away over time. So the model keeps both the direction of trades and how long the impact lasts.

Course reference: we use the transitory impact model shown in the course (see slide page 133 of the lecture notes).

Transitory impact model

Backward definition of the price: the price is the sum of past impacts (Bouchaud):

$$p_t = p_{-\infty} + \sum_{s=-\infty}^{t-1} G(t-s) \varepsilon_s S_s V_s^r,$$

Figure 27: Bouchaud transitory impact model (course slide page 133).

The slide also explains the model parameters. Here is the same simple interpretation as in the notebook:

V : overall impact size. If V is larger, a single trade moves the price more.

r : how impact grows with volume. If $r < 1$, impact is concave (this is commonly observed in real data).

γ : how fast impact decays. A larger γ means faster decay (shorter memory). A smaller γ means slower decay (longer memory).

In our implementation, we weight past trades with the impact kernel $G(\ell) = \ell^{-\gamma}$ and a lag cutoff L (here $L = 50$).

III. Estimation strategy

We now explain how we estimate the parameters of the transitory impact model. The goal is to recover parameters that control: (i) the size of the impact, and (ii) how fast this impact fades away.

We proceed as follows: short-term price changes can be explained by the cumulative effect of past signed trades. So we build a predictor from past trades (with a decay over time), and we fit the model by minimizing the difference between observed price changes and predicted price changes.

Course reference: this estimation approach follows the TD4 methodology: we choose a parametric form for the impact kernel and we use a regression framework.

We now go step by step.

1) Dependent variable: price increments

We first construct the price increments that will be used as the dependent variable. We define:

$$\Delta p_t = p_t - p_{t-1}.$$

This is what the model tries to explain at a short time scale.

```
[104]: df_est = df_vars.copy()

df_est["delta_price"] = df_est["price"].diff() # we compute price increments from the transaction price
df_est[["time", "price", "delta_price"]].head()
```

	time	price	delta_price
0	0.000202	100.000	NaN
1	0.001070	99.984	-0.016
2	0.001496	100.029	0.045
3	0.003336	99.979	-0.050
4	0.003952	100.060	0.081

Figure 28: Computing Δp_t as first differences of the transaction price.

2) Specification of the impact kernel

To estimate the model, we need a parametric form for the impact decay function. Following TD4, we assume a power-law decay over the lag. We fix a maximum lag L (here $L = 50$), and we define lags:

$$\ell = 1, 2, \dots, L, \quad G(\ell) = \ell^{-\gamma}.$$

So γ controls the decay speed: larger γ means faster decay.

```
[25]: max_lag = 50
lags = np.arange(1, max_lag + 1) # lag structure used for the impact kernel

# parametric power law form of the impact kernel
def impact_kernel(lags, gamma):
    return lags ** (-gamma)
```

Figure 29: Power-law impact kernel $G(\ell) = \ell^{-\gamma}$ with max lag $L = 50$.

3) Building the regression signal from past trades

For a given pair (r, γ) , we build a predictor from past signed trades. We first define the signed trade term :

$$u_t = \varepsilon_t S_t Q_t^r,$$

then we aggregate past values with the kernel:

$$X_t = \sum_{\ell=1}^L \ell^{-\gamma} \varepsilon_{t-\ell} S_{t-\ell} Q_{t-\ell}^r.$$

To avoid confusion with the impact coefficient V , we write traded volume as Q_t .

So r controls how impact grows with volume, and γ controls how impact decays with time.

In the code, we implement this with `shift(1)`: we shift the signed term by ℓ steps and add it with weight $G(\ell)$. Missing values created by shifts are replaced by 0 (so we do not use trades that do not exist in the past).

4) Estimating V by OLS for fixed (r, γ)

Once we have X_t and $Y_t = \Delta p_t$, we fit a simple linear relation:

$$Y_t \approx V X_t.$$

For a fixed (r, γ) , the least-squares estimate of V has a closed form:

$$\hat{V} = \frac{\sum_t X_t Y_t}{\sum_t X_t^2}.$$

Then we compute the mean squared error:

$$\text{MSE} = \frac{1}{n} \sum_t (Y_t - \hat{V} X_t)^2.$$

We keep the parameters that give the smallest MSE.

5) Grid search on (r, γ)

We explore a grid of values for r and γ and keep the best fit (smallest MSE). In the code, the grids are:

$$r \in \{0.0, 0.1, \dots, 1.0\}, \quad \gamma \in \{0.1, 0.2, \dots, 2.0\}.$$

Before the grid search we also clean the estimation sample: we keep only rows with non-missing `delta_price`, `eps`, `spread`, and `volume`, and we keep basic checks like $\varepsilon \in \{-1, +1\}$, `spread` ≥ 0 , `volume` ≥ 0 .

Question D - Bouchaud Price Impact Model

```
[115]: #we only keep trades with known volume here.
df_fit = df_est.copy()
df_fit = df_fit.dropna(subset=["delta_price", "eps", "spread", "volume"]).reset_index(drop=True)

df_fit = df_fit[df_fit["eps"].isin([-1, 1])]
df_fit = df_fit[(df_fit["spread"] >= 0) & (df_fit["volume"] >= 0)]

max_lag = 50 # We keep a reasonable maximum lag for the kernel
lags = np.arange(1, max_lag + 1)

def impact_kernel_power(lags, gamma):
    # impact decay:  $G(L) = L^{-\gamma}$ 
    return lags ** (-gamma)

def build_regressor(df, r, gamma, max_lag=50):
    """
    We build  $X_t$  and we model:  $\text{delta\_price}_t \approx V * X_t$ 
    """
    G = impact_kernel_power(np.arange(1, max_lag + 1), gamma)

    signed_term = df["eps"] * df["spread"] * (df["volume"] ** r)
    X = np.zeros(len(df), dtype=float)
    for l in range(1, max_lag + 1):
        X += G[l - 1] * signed_term.shift(l).fillna(0.0).to_numpy()

    Y = df["delta_price"].to_numpy()
    return X, Y

def estimate_V_ols(X, Y):
    # OLS
    denom = np.dot(X, X)
    if denom == 0:
        return np.nan
    return np.dot(X, Y) / denom

def mse(Y, Y_hat):
    return np.mean((Y - Y_hat) ** 2)

# grid search
r_grid = np.arange(0.0, 1.01, 0.1) # concavity exponent
gamma_grid = np.arange(0.1, 2.01, 0.1) # decay exponent

best = {"mse": np.inf, "V": None, "r": None, "gamma": None} # structure using the best result

for r in r_grid:
    for gamma in gamma_grid:
        X, Y = build_regressor(df_fit, r=r, gamma=gamma, max_lag=max_lag)
        valid = np.isfinite(X) & np.isfinite(Y)
        Xv, Yv = X[valid], Y[valid] # build regression variables

        V_hat = estimate_V_ols(Xv, Yv) # estimate parameters
        if np.isnan(V_hat):
            continue
        Y_hat = V_hat * Xv
        current_mse = mse(Yv, Y_hat)

        if current_mse < best["mse"]: # keep the best parameters
            best = {"mse": current_mse, "V": V_hat, "r": r, "gamma": gamma}

best

[115]: {'mse': np.float64(0.005377107972098853),
'V': np.float64(0.001992121301477238),
'r': np.float64(0.7000000000000001),
'gamma': np.float64(2.0)}
```

Figure 30: Estimation step: cleaning, kernel-based regressor construction, OLS estimate of V , MSE, and grid search over (r, γ) .

We report the best fit in a small summary table (MSE and estimated parameters)

```
[116]: pd.DataFrame([best])
```

[116]:	mse	V	r	gamma
0	0.005377	0.001992	0.7	2.0

Figure 31: Best parameters from the grid search (MSE, V , r , γ).

6) Result and interpretation

From the printed results, we get :

$$\text{MSE} \approx 0.005377, \quad \hat{V} \approx 0.001992, \quad \hat{r} = 0.7, \quad \hat{\gamma} = 2.0.$$

Here, r is below 1, so impact is concave in volume (this is a common empirical feature). Also, γ is high, which means the impact decays fast and is mostly temporary.

This can reflect a liquid market environment, but it can also point to limits in the parametric form or in the lag window we chose.

Conclusion: with a simple parametric specification of Bouchaud's transitory impact model, we obtain estimates for the concavity and the decay of price impact. The model captures key features of market impact, but the decay estimate is at the top of the grid ($\gamma = 2.0$), so this looks like a boundary solution. This suggests the true decay might be even faster or that the model is too restrictive for this dataset.

Question E - Multiresolution Correlation & Hurst Exponent

In this question, we study FX rates with Haar wavelets and the Hurst exponent. We look at correlations at different time scales and how volatility scales with time.

Here we use a new dataset: `Dataset_TD5.xlsx`. It contains high-frequency FX prices. So before doing the analysis, we first need to clean the data and build proper time series.

I. Data Description and Preprocessing (Before Question E)

The dataset contains three FX pairs: GBPEUR, SEKEUR and CADEUR.

At each time step, we do not have one single price. We have **HIGH** and **LOW**. So we first build one representative price series for each pair, and then we compute returns.

We start by reading the raw Excel file and printing the first lines. This is just to understand how the file is organized.

```
[69]: df_td5 = pd.read_excel("../data/Dataset_TD5.xlsx", header=None)
      df_td5.head(10)
```

	0	1	2	3	4	5	6	7	8	9	10
0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	GBPEUR Currency	NaN	NaN	NaN	SEKEUR Currency	NaN	NaN	NaN	CADEUR Currency	NaN	NaN
2	Date	HIGH	LOW	NaN	Date	HIGH	LOW	NaN	Date	HIGH	LOW
3	2016-03-07 08:59:59.990000	1.2932	1.2917	NaN	2016-03-07 08:59:59.990000	0.10725	0.1072	NaN	2016-03-07 08:59:59.990000	0.6842	0.6829
4	2016-03-07 09:15:00	1.294	1.293	NaN	2016-03-07 09:15:00	0.10728	0.10717	NaN	2016-03-07 09:15:00	0.6849	0.6841
5	2016-03-07 09:30:00	1.2943	1.2922	NaN	2016-03-07 09:30:00	0.10726	0.10719	NaN	2016-03-07 09:30:00	0.6844	0.6837
6	2016-03-07 09:45:00	1.293	1.2913	NaN	2016-03-07 09:45:00	0.10728	0.10721	NaN	2016-03-07 09:45:00	0.6844	0.6839
7	2016-03-07 10:00:00	1.2931	1.2921	NaN	2016-03-07 10:00:00	0.10725	0.10719	NaN	2016-03-07 10:00:00	0.684	0.6835
8	2016-03-07 10:15:00	1.2926	1.2921	NaN	2016-03-07 10:15:00	0.10724	0.10718	NaN	2016-03-07 10:15:00	0.6839	0.6836
9	2016-03-07 10:30:00	1.293	1.2906	NaN	2016-03-07 10:30:00	0.10722	0.1072	NaN	2016-03-07 10:30:00	0.684	0.6834

Figure 32: Raw preview of `Dataset_TD5.xlsx`: the three FX pairs are in separate blocks (not a clean table yet).

From this preview, we see the file is not a clean table: the three FX pairs are stored in blocks, and there are empty or unnamed columns. So we reorganize everything into one clean dataframe.

We remove the first 3 rows (they are just labels), we keep only the useful columns, and we rename the columns in a clear way: `Date`, `High`, and `Low` for each FX pair.

We also convert the three date columns into a real datetime format. This is important to keep correct time ordering.

Question E - Multiresolution Correlation & Hurst Exponent

```
[70]: df_clean = df_td5.iloc[3:, [0, 1, 2, 4, 5, 6, 8, 9, 10]] # we remove the first 3 rows of the dataset

df_clean.columns = [
    "Date_GBPEUR", "High_GBPEUR", "Low_GBPEUR",
    "Date_SEKEUR", "High_SEKEUR", "Low_SEKEUR",
    "Date_CADEUR", "High_CADEUR", "Low_CADEUR"
]

df_clean["Date_GBPEUR"] = pd.to_datetime(df_clean["Date_GBPEUR"]) # we rename the column and convert datetime format.
df_clean["Date_SEKEUR"] = pd.to_datetime(df_clean["Date_SEKEUR"])
df_clean["Date_CADEUR"] = pd.to_datetime(df_clean["Date_CADEUR"])

df_clean.shape
df_clean.head()
```

```
[70]:
```

	Date_GBPEUR	High_GBPEUR	Low_GBPEUR	Date_SEKEUR	High_SEKEUR	Low_SEKEUR	Date_CADEUR	High_CADEUR	Low_CADEUR
3	2016-03-07 08:59:59.990	1.2932	1.2917	2016-03-07 08:59:59.990	0.10725	0.1072	2016-03-07 08:59:59.990	0.6842	0.6829
4	2016-03-07 09:15:00.000	1.294	1.293	2016-03-07 09:15:00.000	0.10728	0.10717	2016-03-07 09:15:00.000	0.6849	0.6841
5	2016-03-07 09:30:00.000	1.2943	1.2922	2016-03-07 09:30:00.000	0.10726	0.10719	2016-03-07 09:30:00.000	0.6844	0.6837
6	2016-03-07 09:45:00.000	1.293	1.2913	2016-03-07 09:45:00.000	0.10728	0.10721	2016-03-07 09:45:00.000	0.6844	0.6839
7	2016-03-07 10:00:00.000	1.2931	1.2921	2016-03-07 10:00:00.000	0.10725	0.10719	2016-03-07 10:00:00.000	0.684	0.6835

Figure 33: Clean table: selected columns, new names, and datetime conversion.

Now we build one price per FX pair. We use the mid-price (average of HIGH and LOW):

$$P_t = \frac{\text{HIGH}_t + \text{LOW}_t}{2}.$$

This gives one price per time step. Its also helps reduce microstructure noise compared to using only HIGH or only LOW.

Then we compute simple returns from these average prices:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}.$$

Finally, we drop the missing values created by the return computation.

```
[71]: df_clean["Avg_GBPEUR"] = (df_clean["High_GBPEUR"] + df_clean["Low_GBPEUR"]) / 2
df_clean["Avg_SEKEUR"] = (df_clean["High_SEKEUR"] + df_clean["Low_SEKEUR"]) / 2 # we work with average prices to reduce microstructure noise
df_clean["Avg_CADEUR"] = (df_clean["High_CADEUR"] + df_clean["Low_CADEUR"]) / 2

df_clean[["Avg_GBPEUR", "Avg_SEKEUR", "Avg_CADEUR"]] = \
df_clean[["Avg_GBPEUR", "Avg_SEKEUR", "Avg_CADEUR"]].astype(float)

df_clean["Ret_GBPEUR"] = df_clean["Avg_GBPEUR"].pct_change() # we compute returns from average prices
df_clean["Ret_SEKEUR"] = df_clean["Avg_SEKEUR"].pct_change()
df_clean["Ret_CADEUR"] = df_clean["Avg_CADEUR"].pct_change()

df_clean = df_clean.dropna()

print(df_clean)
```

Figure 34: Building mid-prices and computing returns (pct_change())

```

      Date_GBPEUR High_GBPEUR Low_GBPEUR      Date_SEKEUR \
4      2016-03-07 09:15:00      1.294      1.293 2016-03-07 09:15:00
5      2016-03-07 09:30:00      1.2943     1.2922 2016-03-07 09:30:00
6      2016-03-07 09:45:00      1.293     1.2913 2016-03-07 09:45:00
7      2016-03-07 10:00:00     1.2931     1.2921 2016-03-07 10:00:00
8      2016-03-07 10:15:00     1.2926     1.2921 2016-03-07 10:15:00
...
12927 2016-09-07 17:00:00     1.1879     1.1867 2016-09-07 17:00:00
12928 2016-09-07 17:15:00     1.1883     1.1874 2016-09-07 17:15:00
12929 2016-09-07 17:30:00     1.188     1.1874 2016-09-07 17:30:00
12930 2016-09-07 17:45:00     1.1874     1.1866 2016-09-07 17:45:00
12931 2016-09-07 18:00:00     1.187     1.1869 2016-09-07 18:00:00

      High_SEKEUR Low_SEKEUR      Date_CADEUR High_CADEUR Low_CADEUR \
4      0.10728     0.10717 2016-03-07 09:15:00     0.6849     0.6841
5      0.10726     0.10719 2016-03-07 09:30:00     0.6844     0.6837
6      0.10728     0.10721 2016-03-07 09:45:00     0.6844     0.6839
7      0.10725     0.10719 2016-03-07 10:00:00     0.684     0.6835
8      0.10724     0.10718 2016-03-07 10:15:00     0.6839     0.6836
...
12927 0.10536     0.10531 2016-09-07 17:00:00     0.6897     0.6893
12928 0.10537     0.10534 2016-09-07 17:15:00     0.6902     0.6895
12929 0.10538     0.10536 2016-09-07 17:30:00     0.6902     0.6898
12930 0.10537     0.10536 2016-09-07 17:45:00     0.6902     0.6901
12931 0.10537     0.10537 2016-09-07 18:00:00     0.6901     0.6901

      Avg_GBPEUR Avg_SEKEUR Avg_CADEUR Ret_GBPEUR Ret_SEKEUR \
4      1.29350     0.107225     0.68450     0.000812     0.000000e+00
5      1.29325     0.107225     0.68405     -0.000193     -1.110223e-16
6      1.29215     0.107245     0.68415     -0.000851     1.865237e-04
7      1.29260     0.107220     0.68375     0.000348     -2.331111e-04
8      1.29235     0.107210     0.68375     -0.000193     -9.326618e-05
...
12927 1.18730     0.105335     0.68950     -0.000589     1.424231e-04
12928 1.18785     0.105355     0.68985     0.000463     1.898704e-04
12929 1.18770     0.105370     0.69000     -0.000126     1.423758e-04
12930 1.18700     0.105365     0.69015     -0.000589     -4.745184e-05
12931 1.18695     0.105370     0.69010     -0.000042     4.745409e-05

      Ret_CADEUR
4      1.389803e-03
5      -6.574142e-04
6      1.461881e-04
7      -5.846671e-04
8      -1.110223e-16
...
12927 3.627131e-04
12928 5.076142e-04
12929 2.174386e-04
12930 2.173913e-04
12931 -7.244802e-05

[12928 rows x 15 columns]

```

Figure 35: Final cleaned dataset

At this point, preprocessing is done. We now have clean FX return series at the smallest available time resolution, so we can move to the rest of Question E.

II. Multiresolution correlations using Haar wavelets (Question E.a)

In this part, we look at how the three FX rates move together depending on the time scale. Using Haar wavelets, we compute multiresolution correlations between GBPEUR, SEKEUR and CADEUR, and we discuss whether an Epps effect appears.

The idea is to separate very short-term moves from smoother, longer-term moves. Then we compute correlations scale by scale (level by level), instead of using only one single correlation.

Course reference: we use the multiresolution framework based on the Haar wavelet transform (see lecture notes page 291).

Multiresolution analysis

The Haar wavelet allows an intuitive understanding of wavelets:

- It is a Daubechies wavelet with a single zero moment, so very rudimentary to approximate regular functions.
- The scaling function is $\phi = \mathbf{1}_{[0,1]}$ and the wavelet:

$$\psi(t) = \begin{cases} -1 & \text{if } 0 \leq t < 1/2 \\ 1 & \text{if } 1/2 \leq t < 1 \\ 0 & \text{otherwise.} \end{cases}$$

- The decomposition on ϕ and ψ of a function $a\mathbf{1}_{[0,1/2]} + b\mathbf{1}_{[1/2,1]}$ allows us to understand multiresolution analysis.
- The approximation of daily returns r_t at the 2^j scale, i.e. $\sum_{k \in \mathbb{Z}} \langle r, \phi_{j,k} \rangle \phi_{j,k}(t)$, is therefore based on returns of horizon 2^j calculated each day, with overlap.
- The detail is easily calculated on a date as a difference in returns. It is zero if the returns follow a trend, non-zero if they oscillate around an average.

Figure 36: Haar wavelet and multiresolution analysis idea (course slide page 291).

Before computing correlations, we first check that the three FX series are aligned. This is important: correlations only make sense if we compare the same timestamps.

```
[72]: # before we computing correlations, we quickly check that the three FX series are aligned
# on the same timestamps (row by row). If not, we would need to merge on a common time index.
aligned_gbp_sek = (df_clean["Date_GBPEUR"].values == df_clean["Date_SEKEUR"].values).all()
aligned_gbp_cad = (df_clean["Date_GBPEUR"].values == df_clean["Date_CADEUR"].values).all()

print("GBPEUR vs SEKEUR aligned:", aligned_gbp_sek)
print("GBPEUR vs CADEUR aligned:", aligned_gbp_cad)

GBPEUR vs SEKEUR aligned: True
GBPEUR vs CADEUR aligned: True
```

Figure 37: Check that the three date columns are aligned (same timestamps).

We then take the three return series and drop missing values so they stay perfectly aligned: `Ret_GBPEUR`, `Ret_SEKEUR`, and `Ret_CADEUR`.

Now we apply a Haar multiresolution decomposition. The function builds detail coefficients at each level. At each level, we take pairs of points and compute: an average part (smooth move) and a detail part (local fluctuation). Then we keep going on the average part to go to a coarser scale.

After that, for each level, we compute correlations between the three FX pairs using the detail coefficients. To compare levels fairly, we standardize each level series(mean 0, std 1) before computing correlations.

Remark: the exact Haar normalization is not important here, since we standardize the coefficients at each level before computing correlations.

```
[86]: # We take the clean 15min returns and drop NaN so the three series stay perfect aligned
returns = df_clean[["Ret_GBPEUR", "Ret_SEKEUR", "Ret_CADEUR"]].dropna(how="any")

def haar_wavelet_transform(x, levels=3):

    details_per_level = []
    current = x.copy()

    for _ in range(levels):
        # Haar works on pairs, so we just trim the last point if the length is odd
        current = current[: (len(current) // 2) * 2]

        # Haar step: average = approximation, difference = detail
        approx = (current[::2] + current[1::2]) / 2
        detail = (current[::2] - current[1::2]) / 2

        details_per_level.append(detail)
        current = approx # and we keep going on the smoother part

    return details_per_level

# 1 we compute the Haar detail coefficients for each FX return series
wavelet_results = {}
for col in returns.columns:
    wavelet_results[col] = haar_wavelet_transform(returns[col].values, levels=3)

# 2/ Level by Level, we compute correlations between the detail coefficients
correlations = {}
n_levels = len(wavelet_results["Ret_GBPEUR"])

for level in range(n_levels):
    gbp = wavelet_results["Ret_GBPEUR"][level]
    sek = wavelet_results["Ret_SEKEUR"][level]
    cad = wavelet_results["Ret_CADEUR"][level]
    # We standardize (mean 0, std 1) so we compare levels fairly
    gbp = (gbp - gbp.mean()) / gbp.std()
    sek = (sek - sek.mean()) / sek.std()
    cad = (cad - cad.mean()) / cad.std()

    label = f"Level {level + 1}"
    correlations[label] = {
        "GBPEUR-SEKEUR": float(np.corrcoef(gbp, sek)[0, 1]),
        "GBPEUR-CADEUR": float(np.corrcoef(gbp, cad)[0, 1]),
        "SEKEUR-CADEUR": float(np.corrcoef(sek, cad)[0, 1]),
    }

# 3/Finally, we print the results in a clean way
for lvl, corr in correlations.items():
    print(f"{lvl} correlations:")
    for pair, val in corr.items():
        print(f"    {pair}: {val:.4f}")

Level 1 correlations:
GBPEUR-SEKEUR: 0.1601
GBPEUR-CADEUR: 0.2182
SEKEUR-CADEUR: 0.1742
Level 2 correlations:
GBPEUR-SEKEUR: 0.1400
GBPEUR-CADEUR: 0.3276
SEKEUR-CADEUR: 0.2537
Level 3 correlations:
GBPEUR-SEKEUR: 0.2997
GBPEUR-CADEUR: 0.2260
SEKEUR-CADEUR: 0.1600
```

Figure 38: Haar multiresolution decomposition and correlations by level.

We read the levels in time units. With 15-minute data, Level j roughly corresponds to a

horizon of about $(2^j) \times 15$ minutes: Level 1 ≈ 30 minutes, Level 2 ≈ 1 hour, Level 3 ≈ 2 hours. So higher levels mean a coarser time scale.

Here are the correlations we get:

Level 1: correlations are small overall: about 0.16 for GBPEUR-SEKEUR, about 0.17 for SEKEUR-CADEUR, and about 0.22 for GBPEUR-CADEUR. So at the shortest horizon, the three series do not move together that strongly.

Level 2: dependence becomes more visible for some pairs. GBPEUR-CADEUR increases to about 0.33 and SEKEUR-CADEUR rises to about 0.25, while GBPEUR-SEKEUR stays lower around 0.14.

Level 3: the ranking changes again. GBPEUR-SEKEUR becomes the most correlated pair at about 0.30, while GBPEUR-CADEUR goes back to about 0.23 and SEKEUR-CADEUR drops to about 0.16. So the pattern is not perfectly monotone across levels, which is normal: each level looks at a different time scale, not just a simple aggregation.

A simple way to interpret this is through the usual Epps effect. At very short horizons, correlations often look weaker. They become easier to see once we move to slightly longer time scales.

Conclusion: overall, correlations look weaker at the very highest frequency, and they become clearer when we move to coarser scales. We also see that the most correlated pair can change across levels, which shows that dependence is scale-dependent.

We now move to the Hurst exponent estimation to study the scaling behavior of volatility.

III. Estimation of Hurst exponent and volatility scaling (Question E.b)

In this second part, we study the scaling behavior of the three FX return series. We estimate the Hurst exponent for GBPEUR, SEKEUR and CADEUR. Then we use these Hurst estimates to compute annualized volatilities in a way that does not rely on independence.

Course reference: the Hurst estimator used here is recalled in the lecture notes (pages 251-252).

An estimate of H for a known Gaussian volatility

We define empirical absolute moments on our sample $[0, T]$ with $1/N$ spacings:

$$M_k = \frac{1}{NT} \sum_{i=1}^{NT} |X(i/N) - X((i-1)/N)|^k. \quad (2)$$

M_k is an estimate of $\mathbb{E}(|X(.) - X(. - 1/N)|^k)$, which value is:

$$\mathbb{E}(|X(.) - X(. - 1/N)|^k) = \frac{2^{k/2} \Gamma(\frac{k+1}{2})}{\Gamma(\frac{1}{2})} \sigma^k N^{-kH}. \quad (3)$$

It thus leads to the following estimate of H :

$$\check{H} = - \frac{\log(\sqrt{\pi} M_k / [2^{k/2} \Gamma(\frac{k+1}{2}) \sigma^k])}{k \log(N)},$$

which converges towards H at the rate $O((\sqrt{\varepsilon_N} \log(N))^{-1})$, when N goes to infinity. Whatever k , it depends on σ .

Figure 39: Estimation of H from absolute moments (Course slide pages 251-252).

From now $k = 2$ (estimator then stems from $\mathbb{E}\{(B_H(t) - B_H(s))^2\} = \sigma^2 |t - s|^{2H}$). We combine it with the same kind of statistic M'_2 based on observations in the same window but with halved resolution:

$$M'_2 = \frac{2}{NT} \sum_{i=1}^{NT/2} |X(2i/N) - X(2(i-1)/N)|^2.$$

Then:

$$\hat{H} = \frac{1}{2} \log_2 \left(\frac{M'_2}{M_2} \right). \quad (4)$$

$\hat{H}$ converges almost surely toward H .

Figure 40: Final estimator used here (with $k = 2$) (Course slide pages 251-252).

In practice, we implement the estimator with $k = 2$. We compute two empirical moments: one using step size 1 (all consecutive points), and one using step size 2 (every two points). Then we combine them to get an estimate of H .

Important detail: the estimator is defined for a price-like process X_t . Here we start from returns, so we build a price-like series by cumulating returns:

$$X_t = \sum_{i=1}^t r_i.$$

This gives a convenient proxy of a log-price path over the sample.

We now compute H for the three FX series.

```
[84]: def Hurst_Exponent(X):
    k = 2 # we use the moment of order 2 (same estimator as in the slides)
    T = len(X)

    # quick safety check (we need enough points)
    if T < 4:
        return np.nan

    # 1/we compute M_2 with step size = 1
    M_2 = 0
    for i in range(1, T):
        M_2 += abs(X[i] - X[i - 1])**k
    M_2 /= (T - 1)

    # 2/ we compute M_2_prime with step size = 2
    M_2_prime = 0
    for i in range(1, T // 2):
        M_2_prime += abs(X[2 * i] - X[2 * i - 2])**k
    M_2_prime /= (T // 2 - 1)

    # 3/we apply H = 0.5 * log2(M_2_prime / M_2)
    return 0.5 * np.log2(M_2_prime / M_2)

# we take the 15-min returns from our cleaned dataset
ret_gbp = returns["Ret_GBPEUR"].values.astype(float)
ret_sek = returns["Ret_SEKEUR"].values.astype(float)
ret_cad = returns["Ret_CADEUR"].values.astype(float)

# we build X_t by cumulating returns (so it behaves like a price-like process)
X_GBPEUR = np.cumsum(ret_gbp)
X_SEKEUR = np.cumsum(ret_sek)
X_CADEUR = np.cumsum(ret_cad)

# we compute H for each FX rate
Hurst_GBPEUR = Hurst_Exponent(X_GBPEUR)
Hurst_SEKEUR = Hurst_Exponent(X_SEKEUR)
Hurst_CADEUR = Hurst_Exponent(X_CADEUR)

print("Hurst exponent for GBPEUR:", Hurst_GBPEUR)
print("Hurst exponent for SEKEUR:", Hurst_SEKEUR)
print("Hurst exponent for CADEUR:", Hurst_CADEUR)

Hurst exponent for GBPEUR: 0.686860136564295
Hurst exponent for SEKEUR: 0.6810236267055336
Hurst exponent for CADEUR: 0.6810811380086749
```

Figure 41: Hurst exponent estimation for the three FX series (implementation of the $k = 2$ estimator).

We obtain Hurst exponents all above 0.5: around 0.6869 for GBPEUR, 0.6810 for SEKEUR,

and 0.6811 for CADEUR. So the three values are close and suggest persistence compared to the Brownian benchmark ($H = 0.5$).

Course reminder: for a fractional Brownian motion interpretation: $H = 0.5$ corresponds to standard Brownian motion, $H > 0.5$ means persistent increments, and $H < 0.5$ means anti-persistent increments (lecture notes page 210).

Increments of B_H of duration τ are stationary: centred Gaussian of variance $\sigma^2|\tau|^{2H}$.

- if $H = 1/2$, the fBm is identical to a standard Brownian motion,
- if $H > 1/2$, the increments are persistent,
- if $H < 1/2$, the increments are anti-persistent.

Figure 42: Interpretation of H (Brownian / persistent / anti-persistent) (course slide page 210).

Notes: we interpret these results with caution. This is intraday data, the sample is finite, and microstructure effects can affect scaling estimates. Also, we estimate H on the cumulative sum of returns (a price-like proxy), not directly on log-prices.

Annualized volatility using Hurst scaling

We now compute annualized volatilities using the Hurst scaling law, instead of the usual square-root-of-time rule.

Course reference: the scaling law is:

$$\sigma(\Delta t) \propto (\Delta t)^H,$$

so when we go from Δt to T , we scale volatility like:

$$\sigma(T) = \sigma(\Delta t) \left(\frac{T}{\Delta t} \right)^H.$$

We use this because it does not assume independent returns (lecture notes, page 210).

Here the base step is 15 minutes. To annualize, we use the factor:

$$N = 252 \times 10 \times 4,$$

because we take 252 trading days per year, about 10 trading hours per day, and 4 points per hour (15-minute data).

Remark: N is an approximation (we use about 10 trading hours per day). A slightly different choice would change the annualized values a bit, but not the overall comparison across assets.

So:

$$\sigma_{\text{annual}} = \sigma_{15\text{min}} N^H.$$

We compute $\sigma_{15\text{min}}$ as the standard deviation of 15-minute returns on the sample, then we apply the scaling with the estimated H .

```
[106]: def calculate_annualized_volatility(returns_series, hurst_exponent):
# first we just take the average return on our sample
mean_return = sum(returns_series) / len(returns_series)

# then we compute the volatility at the 15-min step (simple std formula)
std_return = np.sqrt(
    sum((x - mean_return) ** 2 for x in returns_series) / (len(returns_series)))

# now we scale it up to one year using the Hurst exponent (so we don't rely on independence)
# for our dataset, we take 252 trading days per year and about 10 trading hours per day
# with 15-min data, that's 4 points per hour
return std_return * (252 * 10 * 4) ** hurst_exponent

#we apply the same formule to each FX returns series
annually_vol_GBPEUR = calculate_annualized_volatility(
    returns["Ret_GBPEUR"].dropna().values.astype(float),
    Hurst_GBPEUR)
print(f"Annualized volatility for GBPEUR: {annually_vol_GBPEUR:.6f}")

annually_vol_SEKEUR = calculate_annualized_volatility(
    returns["Ret_SEKEUR"].dropna().values.astype(float),
    Hurst_SEKEUR)
print(f"Annualized volatility for SEKEUR: {annually_vol_SEKEUR:.6f}")

annually_vol_CADEUR = calculate_annualized_volatility(
    returns["Ret_CADEUR"].dropna().values.astype(float),
    Hurst_CADEUR)
print(f"Annualized volatility for CADEUR: {annually_vol_CADEUR:.6f}")

Annualized volatility for GBPEUR: 0.350331
Annualized volatility for SEKEUR: 0.174243
Annualized volatility for CADEUR: 0.269808
```

Figure 43: Annualized volatility using Hurst-based scaling (15-minute step scaled to one year).

We obtain annualized volatilities of about: 0.3503 for GBPEUR, 0.1742 for SEKEUR, and 0.2698 for CADEUR.

Conclusion (E.b): we find similar Hurst exponents for the three FX series, all above 0.5. This suggests some persistence, but we keep in mind the limits of intraday data and the estimation method. Using H -based scaling then allows us to annualize volatility beyond the standard square-root-of-time rule.

Overall, these results show that FX behavior depends on the time scale, so time scale should be taken into account when analyzing dependence and risk.

General conclusion (key results)

Question	Main results
A (VaR)	Empirical 1-day VaR ($\alpha = 0.05$): $\text{VaR} \approx -0.037779 \Rightarrow \approx 3,777.91$ euros (portfolio 100,000).
A(KDE VaR)	Kernel VaR (biweight KDE): $\text{VaR} \approx -0.034786 \Rightarrow \approx 3,478.61$ euros; out-of-sample exceedances (2017–2018): $9/510 \approx 1.76\%$.
B (ES)	Expected Shortfall ($\alpha = 0.05$): $\text{ES} \approx -0.05004$.
C (EVT)	Pickands tail index: $\xi_{\text{gains}} \approx 0.5772$ (heavy tail), $\xi_{\text{losses}} \approx -0.5090$ (lighter tail); EVT VaR (losses): $90\% \approx 0.03141$, $95\% \approx 0.04354$, $99\% \approx 0.05958$.
D (Impact)	Bouchaud model estimates: $r \approx 0.7$, $\gamma \approx 2.0$, $V \approx 0.001992$, $\text{MSE} \approx 0.005377$.
E (Wavelets)	Haar multiresolution correlations: weaker at the finest scale, clearer at coarser scales (Epps-type effect).
E (Hurst)	Hurst: $H_{\text{GBPEUR}} \approx 0.6869$, $H_{\text{SEKEUR}} \approx 0.6810$, $H_{\text{CADEUR}} \approx 0.6811$; Annualized vol (H-scaling): $\text{GBPEUR} \approx 0.3503$, $\text{SEKEUR} \approx 0.1742$, $\text{CADEUR} \approx 0.2698$.